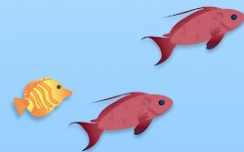


Wakacyjne Wyzwanie

Z  KN Solvro

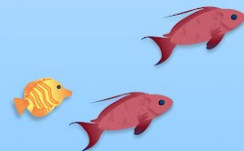


Wykład 6

Monitoring: Grafana i Prometheus

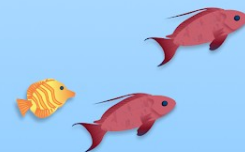


Plan na dziś



- Wstęp
 - Czyli ogółem o softcie
- Instalacja i konfiguracja Grafany + Prometheusa
- Co można monitorować Prometheusem?
- Kwerendy PromQL
 - Czyli jak tworzyć własne dashboardy
- Tworzenie alertów



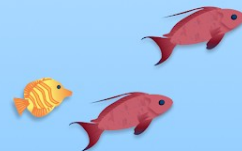


Rozdział 1

Wstęp



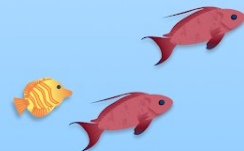
Po co monitorować?



- W poprzednich tygodniach mówiliśmy o stawianiu usług
 - Stawianie usług to tylko początek...
 - Usługi trzeba utrzymywać
- **Gdy jakiś serwis padnie, kto pierwszy się o tym dowie?**
 - Bez monitoringu, rolę systemu monitorowania efektywnie pełnią użytkownicy
 - Co, jeżeli coś padnie o 21:37 w sobotę?
- **Co jeżeli jakiś serwis zacznie działać nieprawidłowo?**
 - np. będzie zużywał nadmierną ilość zasobów
 - Bez monitoringu, dopóki serwis działa, nikt nie będzie wiedział o awarii
 - Co gorsza, nikt nie będzie wiedział, kiedy awaria się zaczęła



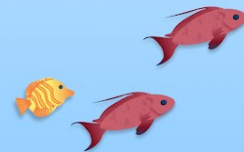
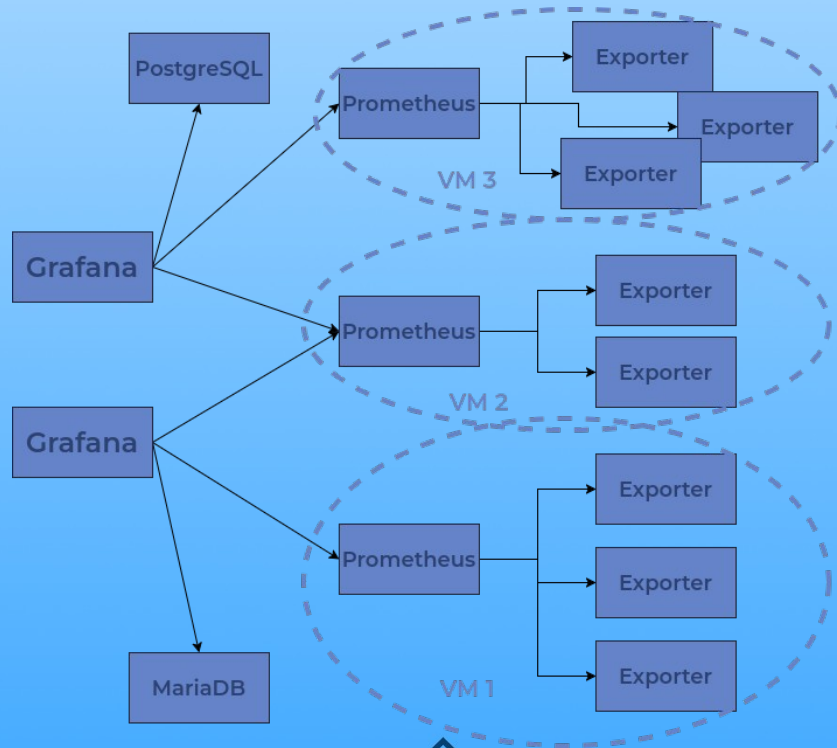
Grafana i Prometheus



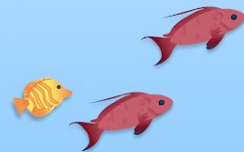
- **Grafana to frontend do baz metryk**
 - Wizualizuje dane w przystępny sposób
 - Potrafi generować alerty i wysyłać powiadomienia
- **Prometheus to baza dla danych, które zmieniają się w czasie**
 - Pobiera metryki ze skonfigurowanych serwisów
 - Przechowuje pobrane metryki przez określony czas
 - Wystawia API pozwalające na odpytywanie bazy metryk
 - Potrafi generować alerty
- **Exportery to serwisy, z których Prometheus pobiera metryki**
 - „Eksportują” informacje z infrastruktury do Prometheusa
 - ▶ np. zużycie zasobów systemowych, stan kontenerów, stan serwisów/hostów, statystyki serwisów
 - Mogą być dodatkowym serwisem, lub wbudowane w monitorowany serwis



Grafana i Prometheus



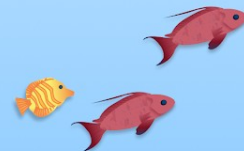
Czym są metryki?



- Każda metryka w Prometheus to dana liczbową opisująca działanie systemu/aplikacji
 - np. ilość RAMu zużywanego przez aplikację, czas spędzony przez procesor w określonym stanie, czy jakiś host jest dostępny przez sieć (0 nie/1 tak)
- Każda metryka składa się z nazwy, etykiet i wartości
 - Nazwa określa przedmiot pomiaru (np. zużycie RAMu)
 - Etykieta określa podmiot pomiaru (którego hosta/kontenera/instancji dotyczył pomiar)
 - Wartość to wynik pomiaru (np. ile RAMu zużywał konkretny kontener)



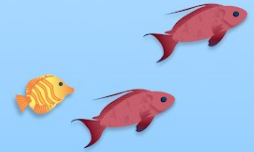
Przykład metryk



node_memory_Active_bytes{instance="1:9100", job="node-exporter"}	69953327104
node_memory_Active_bytes{instance="11:9100", job="node-exporter"}	556863488
node_memory_Active_bytes{instance="12:9100", job="node-exporter"}	7501721600
node_memory_Active_bytes{instance="12:9100", job="node-exporter"}	742117376
node_memory_Active_bytes{instance="14:9100", job="node-exporter"}	4576595968
node_memory_Active_bytes{instance="15:9100", job="node-exporter"}	3799085056
node_memory_Active_bytes{instance="16:9100", job="node-exporter"}	167116800
node_memory_Active_bytes{instance="18:9100", job="node-exporter"}	1814634496
node_memory_Active_bytes{instance="19:9100", job="node-exporter"}	586313728
node_memory_Active_bytes{instance="13:9100", job="node-exporter"}	1055342592

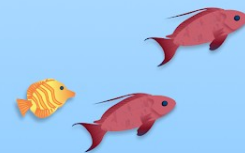


Typy metryk



- „Counter” – licznik, wartość która tylko się zwiększa
 - np. liczba obsłużonych żądań od uruchomienia serwisu
- „Gauge” – miernik, wartość która może się zmniejszać
 - np. zużycie RAMu
- Histogram – wiele liczników, które zliczają „obserwacje” według zakresów wartości
 - np. liczba żądań, których czas przetwarzania mieści się w określonych zakresach
- „Summary” – „podsumowanie histogramu”, wartości danych „kwantyli”
 - np. czas przetwarzania żądań dla przeciętnego żądania (mediana, q0.5), dla najgorszych 1% żądań (q0.99), dla najlepszych 10% żądań (q0.1)...
- Więcej informacji: prometheus.io/docs/concepts/metric_types/



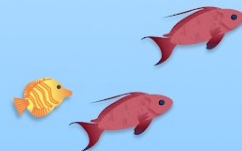


Rozdział 2

Instalacja



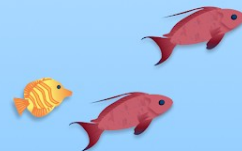
Skąd wziąć grafanę/prometheusa?



- Dostępne są zarówno pakiety w repozytoriach dystrybucji, jak i obrazy kontenerów
- Grafanę zazwyczaj stawiam w kontenerze
- Prometheusa różnie
 - Prometheus często musi zbierać dane spoza kontenerów, lub z różnych sieci dockerowych
 - Dlatego wygodnie jest go postawić na hostcie



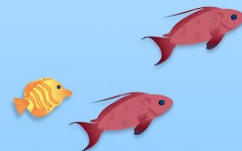
Jak wystawiać eksportery metryk?



- Często nie chcemy publicznie udostępniać metryk
 - Więc ważne jest, by prometheusa i eksportery wystawiać na adresach lokalnych lub prywatnych
- W przypadku zbierania metryk z tego samego hosta, lubię eksportery wystawiać na jednym z adresów localhost w IPv4, lub na losowej sieci prywatnej IPv6
 - W przypadku sieci prywatnej, ustawiam reguły na firewallu odrzucające ruch z internetu – just in case
- Prometheusa często wystawiam na adresie VPNa, który łączy moje maszyny w klastrze



Instalujemy prometheusa



```
root@debian:~# apt install prometheus
Installing:
  prometheus

Installing dependencies:
  fonts-glyphicons-halflings  libjs-eonasdan-bootstrap-datetimepicker  libsnmp40t64
  freeipmi-common             libjs-jquery                               libtime-duration-perl
  ipmitool                    libjs-jquery-hotkeys                      libtime-date-perl
  javascript-common           libjs-moment                              moreutils
  jq                           libjs-moment-timezone                     node-jquery
  libfreeipmi17               libjs-mustache                             nvme-cli
  libgcrypt20                  libjs-popper.js                           openipmi
  libgpg-error-l10n           libjs-rickshaw                             prometheus-node-exporter
  libgpg-error0               libjs-sizzle                               prometheus-node-exporter-collectors
  libgpm2                      libncurses6                               promtool
  libio-pty-perl              libnvme1t64                               python3-decorator
  libipc-run-perl             libonig5                                   python3-prometheus-client
  libjq1                       libopenipmi0t64                           smartmontools
  libjs-bootstrap             libsensors-config                         uuid-runtime
  libjs-bootstrap4            libsensors5
  libjs-d3                     libsnmp-base

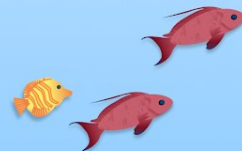
Suggested packages:
  freeipmi-tools  rng-tools  gpm  lm-sensors  snmp-mibs-downloader  gsmartcontrol  smart-notifier  mailx  |  mailutils  gpg

Summary:
  Upgrading: 0, Installing: 47, Removing: 0, Not Upgrading: 0
  Download size: 51.7 MB
  Space needed: 254 MB / 15.8 GB available

Continue? [Y/n]
Get:1 http://security.debian.org/debian-security trixie-security/main amd64 uuid-runtime amd64 2.41.5-0+deb13u1 [60.7 kB]
```



Konfigurujemy prometheusa



```
() 192.168.122.80 — Konsola
Plik Edycja Widok Zakładki Wtyczki Ustawienia Pomoc
Nowa karta Podziel widok Zezwól na śledzenie myszy Skopiuj Wklej Poszukaj

# Alertmanager configuration
alerting:
  alertmanagers:
    - static_configs:
      - targets: ['localhost:9093']

# Load rules once and periodically evaluate them according to the global 'evaluation_interval'.
rule_files:
# - "first_rules.yml"
# - "second_rules.yml"

# A scrape configuration containing exactly one endpoint to scrape:
# Here it's Prometheus itself.
scrape_configs:
# The job name is added as a label 'job=<job_name>' to any timeseries scraped from this config.
- job_name: 'prometheus'

  # Override the global default and scrape targets from this job every 5 seconds.
  scrape_interval: 5s
  scrape_timeout: 5s

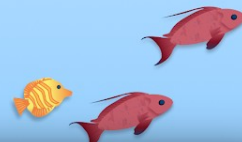
  # metrics_path defaults to '/metrics'
  # scheme defaults to 'http'.

  static_configs:
    - targets: ['localhost:9090']

~
~
~
/etc/prometheus/prometheus.yml 33,0-1 Bot
"/etc/prometheus/prometheus.yml" 38L, 1252B written
```



Instalacja grafany



```
services:
  grafana:
    image: docker.io/grafana/grafana:latest
    volumes:
      - data:/var/lib/grafana
    ports:
      - 127.67.21.37:3000:3000

volumes:
  data:
    ~

compose.yml
root@debian:~/grafana# podman compose up -d
>>> Executing external compose provider "/usr/lib
to disable this message. <<<<

[+] Running 1/1
 ✓ Container grafana-grafana-1 Started
root@debian:~/grafana#
```

term://~/grafana//6783:/bin/bash
-- TERMINAL --

```
() 192.168.122.80 — Konsola

Plik Edycja Widok Zakładki Wtyczki Ustawienia Pomoc
Nowa karta Podziel widok Zezwól na śledzenie myszy Skopiuj Wklej Poszukaj

() 192.168.122.80 x () 192.168.122.70 x root: sudo x

server {
    listen 80;
    listen [::]:80;
    server_name grafana.minibomba.wakacyjne-wyzwanie;

    location / {
        proxy_pass http://127.67.21.37:3000;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Port $remote_port;
        proxy_set_header X-Forwarded-Host $host;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;

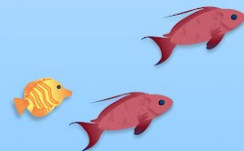
        proxy_http_version 1.1;
        proxy_set_header Connection $http_connection;
        proxy_set_header Upgrade $http_upgrade;
    }
}

~
~
~
~
~
~
/etc/nginx/sites-enabled/grafana
```

12,20-34 All



Konfiguracja grafany



- Trzeba wyciągnąć plik grafana.ini z kontenera

```
root@debian:~/grafana# podman inspect grafana-grafana-1 --format '{{ .GraphDriver.Data.MergedDir }}'  
/var/lib/containers/storage/overlay/01ff1653d966d7383399d0480cf43f41929ee9aae64ef820e5bbd39e3e40a826/merged  
root@debian:~/grafana# cp /var/lib/containers/storage/overlay/01ff1653d966d7383399d0480cf43f41929ee9aae64ef820e5bbd39e3e40a826/m  
erged/etc/grafana/grafana.ini .  
root@debian:~/grafana#
```

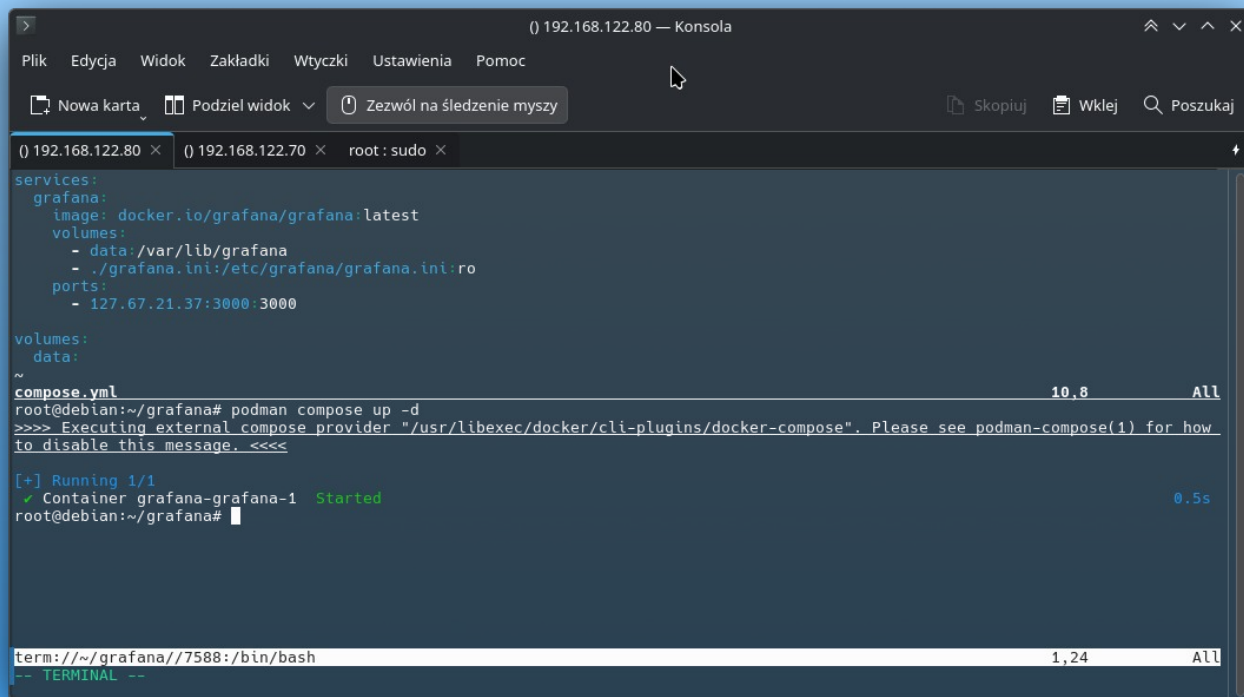
```
# instance name, defaults to HOSTNAME environment variable value or hostname if HOSTNAME var is empty  
instance_name = minibomba.wakacyjne-wyzwanie
```

```
# The public facing domain name used to access grafana from a browser  
domain = grafana.minibomba.wakacyjne-wyzwanie
```

```
# Redirect to correct domain if host header does not match domain  
# Prevents DNS rebinding attacks  
enforce_domain = true
```



Konfiguracja grafany



A terminal window titled "192.168.122.80 — Konsola" with a menu bar (Plik, Edycja, Widok, Zakładki, Wtyczki, Ustawienia, Pomoc) and a toolbar (Nowa karta, Podziel widok, Zezwól na śledzenie myszy, Skopiuj, Wklej, Poszukaj). The terminal shows the configuration of a Grafana service using Docker Compose. The configuration includes the image "docker.io/grafana/grafana:latest", volumes for data and configuration, and ports 3000. The command "podman compose up -d" is executed, resulting in the container "grafana-grafana-1" starting successfully. The terminal also shows a prompt to enter the container's shell.

```
services:
  grafana:
    image: docker.io/grafana/grafana:latest
    volumes:
      - data:/var/lib/grafana
      - ./grafana.ini:/etc/grafana/grafana.ini:ro
    ports:
      - 127.67.21.37:3000:3000

volumes:
  data:

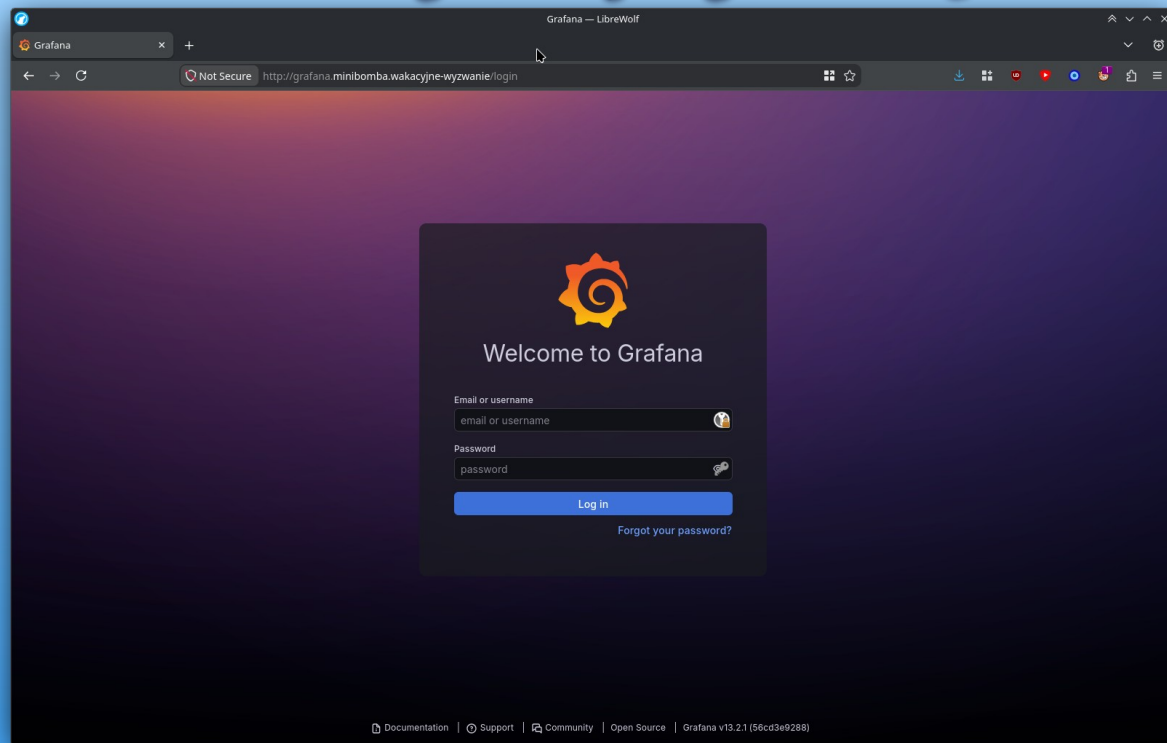
~
compose.yml
root@debian:~/grafana# podman compose up -d
>>>> Executing external compose provider "/usr/libexec/docker/cli-plugins/docker-compose". Please see podman-compose(1) for how
to disable this message. <<<<

[+] Running 1/1
 ✓ Container grafana-grafana-1 Started
root@debian:~/grafana#
```

term://~/grafana//7588:/bin/bash
-- TERMINAL --



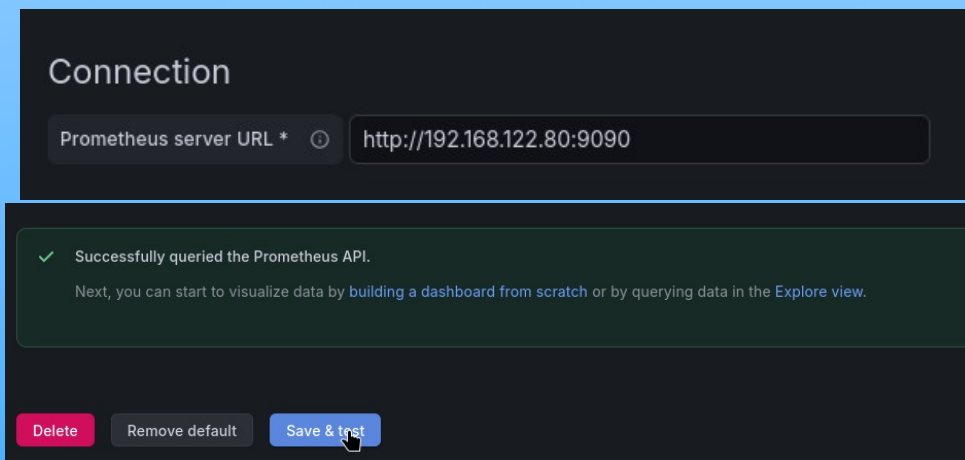
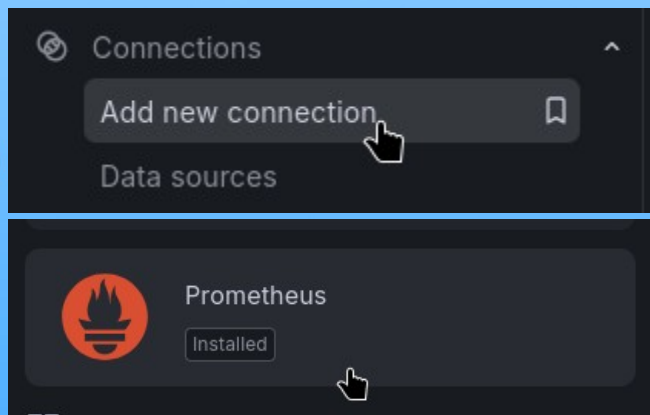
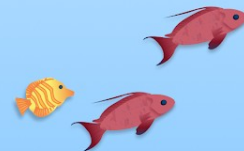
Konfiguracja grafany

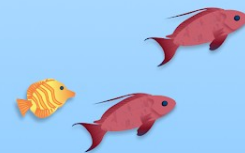


Hasło:
admin/admin
(do zmiany)



Konfiguracja grafany



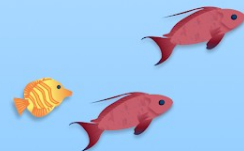


Rozdział 3

Co można monitorować?



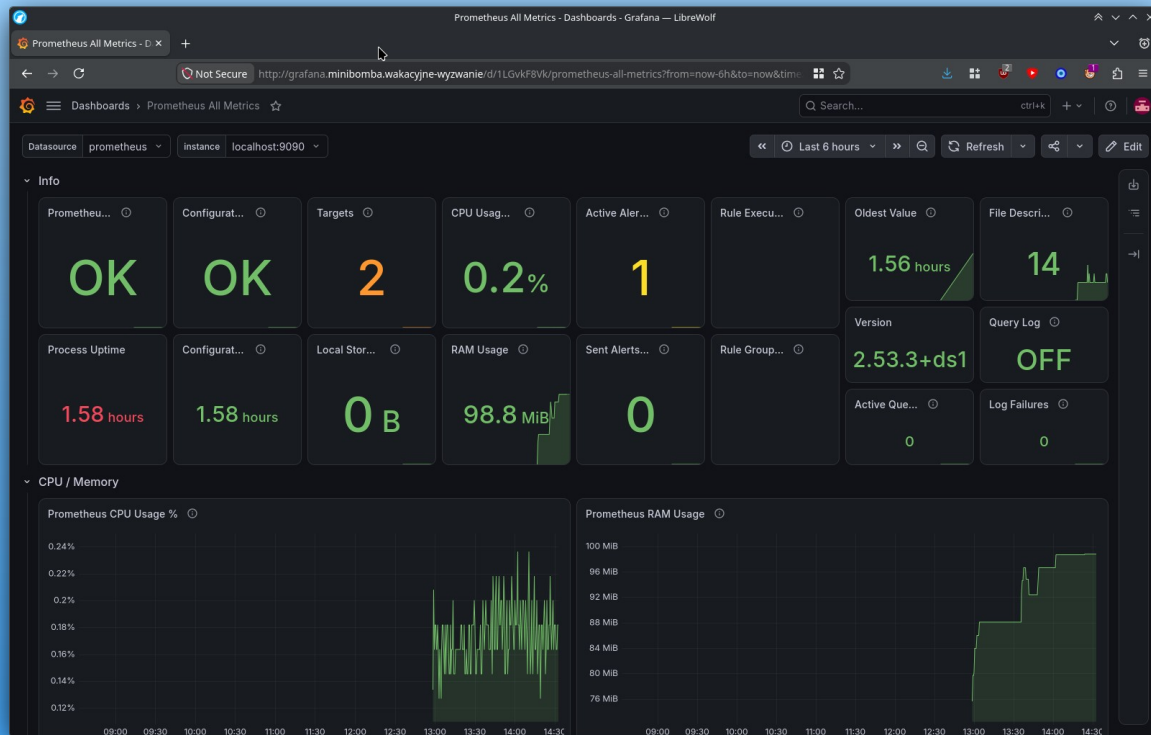
Prometheus



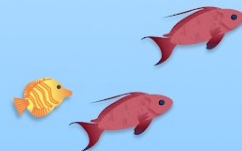
- W domyślnej konfiguracji prometheus zbiera metryki ze samego siebie
- Wystarczy dodać na grafanie odpowiedni dashboard
 - np. grafana.com/grafana/dashboards/19268-prometheus/



Prometheus



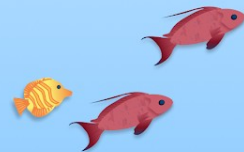
Node exporter



- Node exporter wystawia metryki z zużycia zasobów hosta
 - Trzeba zainstalować na hostcie i dodać do prometheusa
 - Metryki wystawia na porcie 9100
- Dashboard: grafana.com/grafana/dashboards/1860-node-exporter-full/



Node exporter



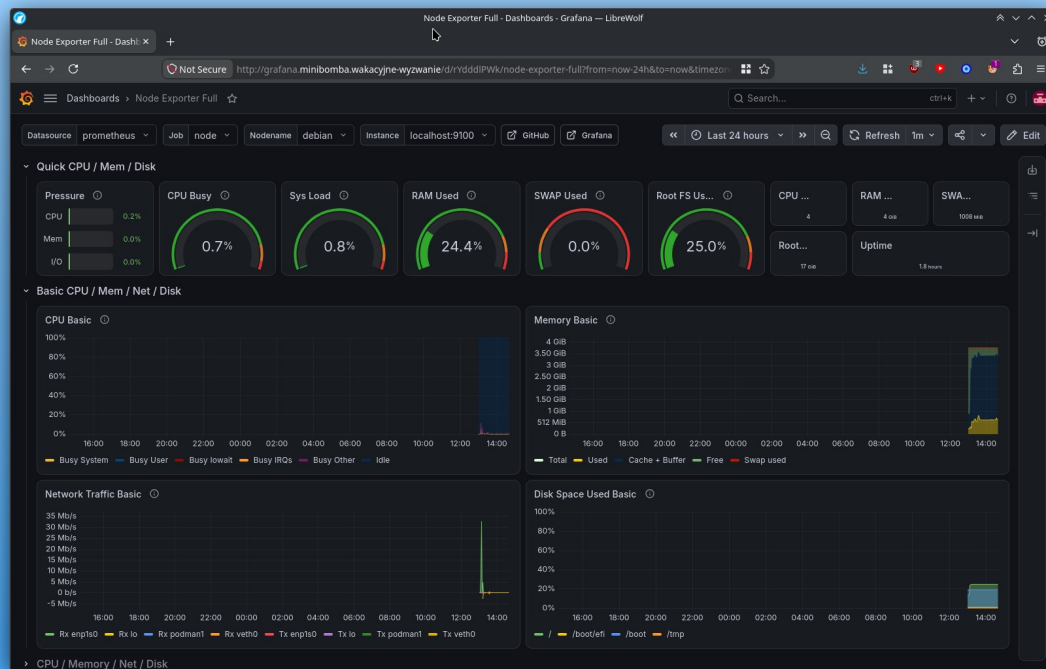
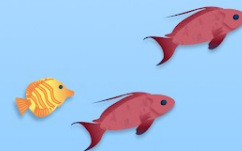
```
root@debian:~# apt install prometheus-node-exporter
prometheus-node-exporter is already the newest version (1.9.0-1+b4).
prometheus-node-exporter set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
root@debian:~#
```

```
- job_name: 'node'
  static_configs:
    - targets: ['localhost:9100']
~
/etc/prometheus/prometheus.yml
"/etc/prometheus/prometheus.yml" 41L, 1329B written
```

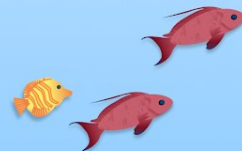
```
root@debian:~# systemctl reload prometheus
root@debian:~#
```



Node exporter



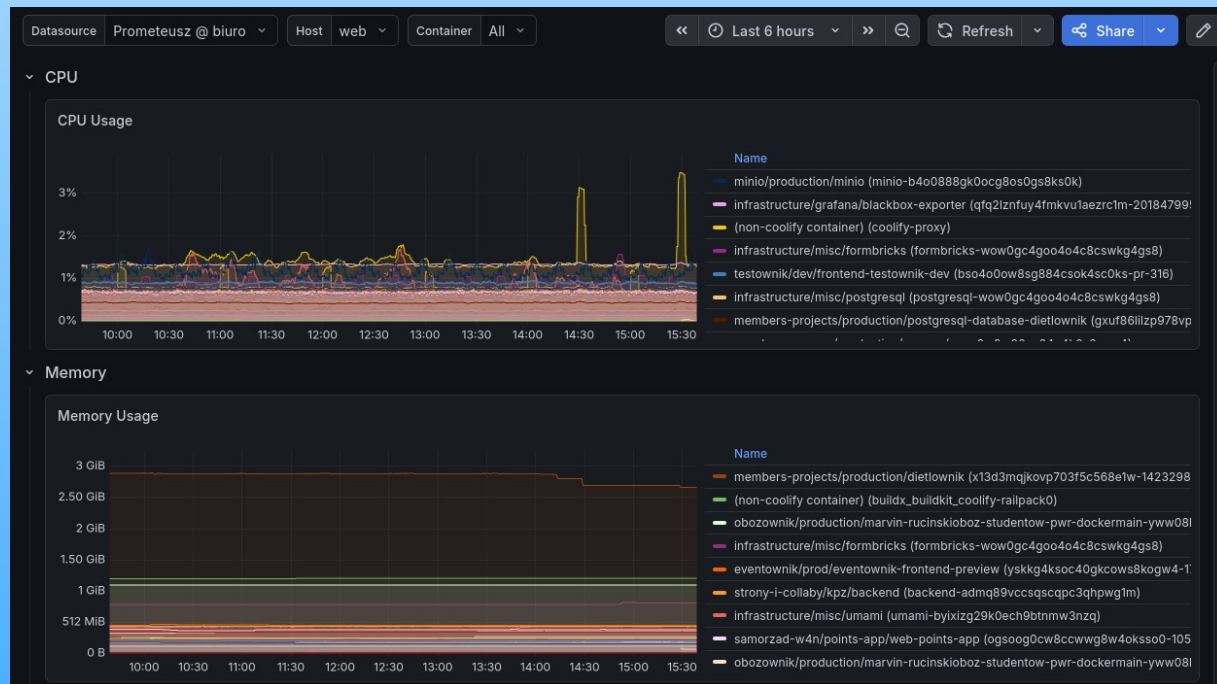
cAdvisor



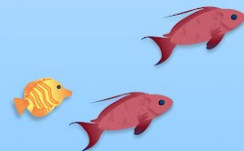
- Eksporter dla metryk ogólnych o kontenerach
 - github.com/google/cadvisor
 - github.com/google/cadvisor/blob/master/docs/storage/prometheus.md
 - grafana.com/grafana/dashboards/14282-cadvisor-exporter/



cAdvisor



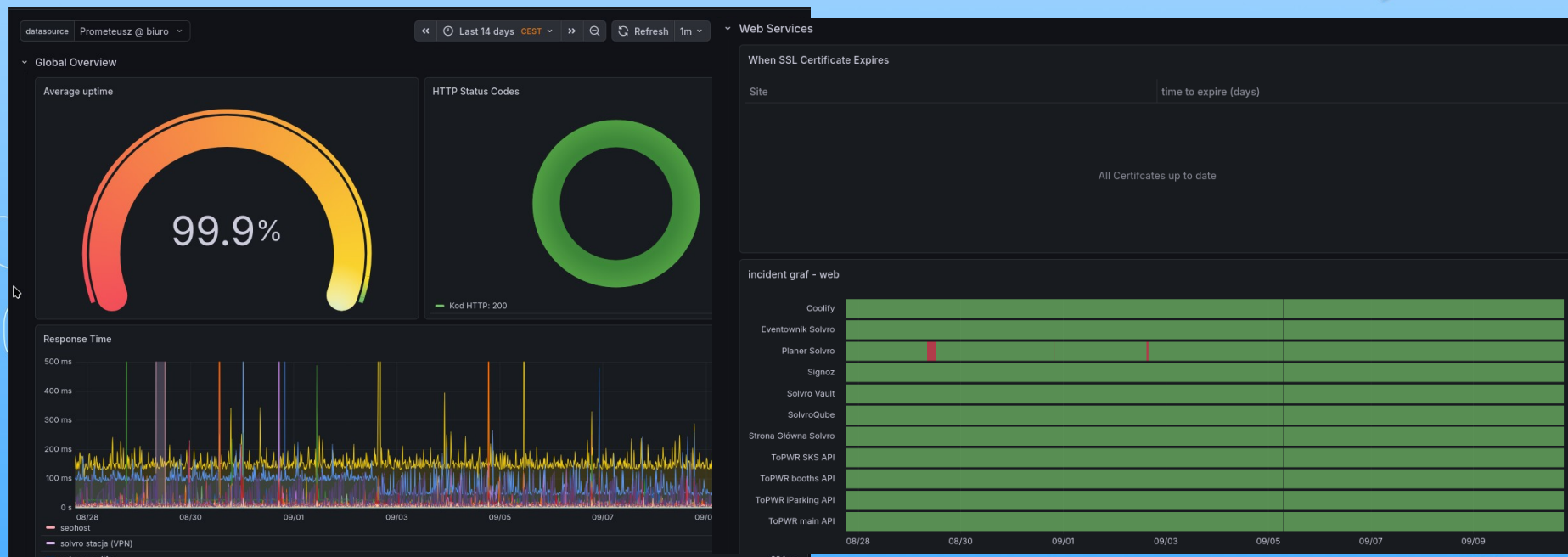
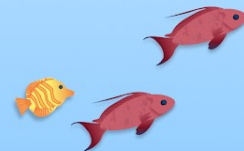
Blackbox exporter



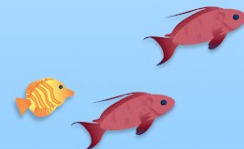
- Pingi ICMP/HTTP jako metryki w Prometheus
 - github.com/prometheus/blackbox_exporter
- Brak dashboarda



Blackbox exporter



Blackbox exporter



Coolify

Coolify

on

Uptime

100.0%

Eventownik Solvro

Eventownik Solvro

on

Uptime

100.0%

Planer Solvro

Planer Solvro

on

Uptime

98.5%

Signoz

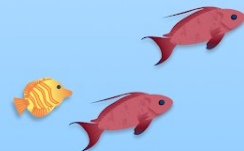
Signoz

on

Uptime

100.0%

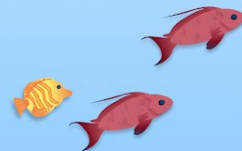




- Prometheus-nginx-exporter
- Brak dashboarda
 - ale domyślnie też mało danych jest



nginx



```
server {  
    listen 127.67.21.37:9092;  
    access_log off;  
    server_name "";  
  
    location /stub_status {  
        stub_status;  
    }  
}  
~  
/etc/nginx/sites-enabled/internal
```

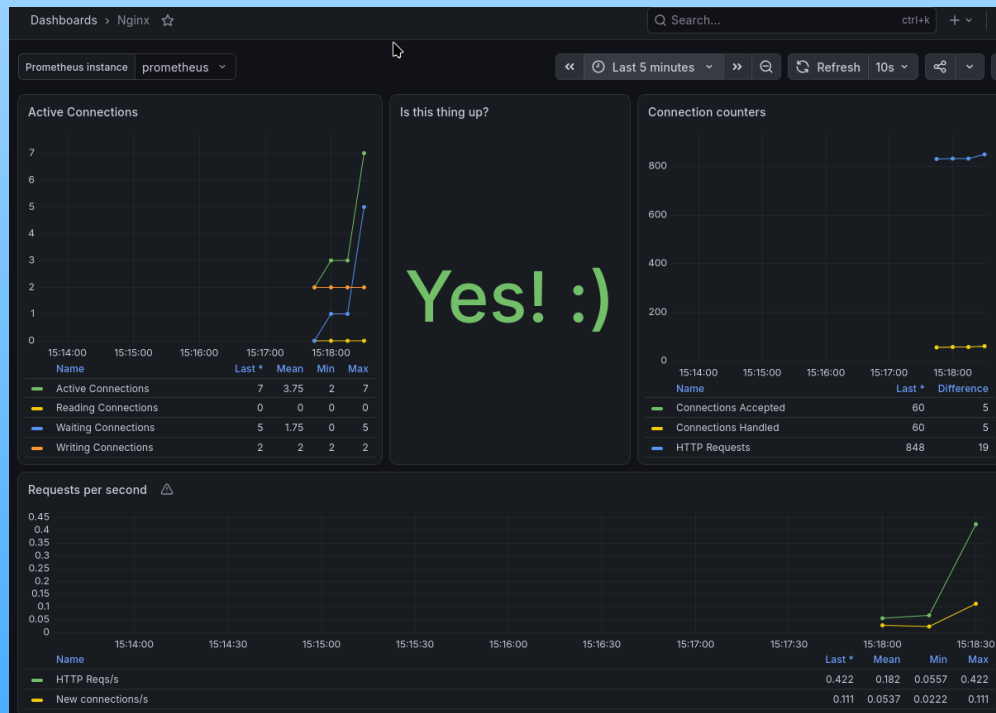
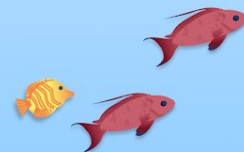
```
- job_name: 'nginx'  
  static_configs:  
    - targets: ['127.67.21.37:9093']  
~  
/etc/prometheus/prometheus.yml
```

```
root@debian:~# systemctl restart prometheus-nginx-exporter.service  
root@debian:~# systemctl reload prometheus  
root@debian:~#
```

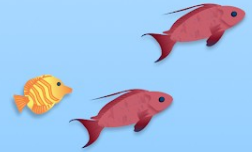
```
# Set the command-line arguments to pass to the server.  
# Due to shell escaping, to pass backslashes for regexes, you need to double  
# them (\\d for \d). If running under systemd, you need to double them again  
# (\\\\d to mean \d), and escape newlines too.  
ARGS="--nginx.scrape-uri=http://127.67.21.37:9092/stub_status --web.listen-address=127.67.21.37:9093"  
~  
/etc/default/prometheus-nginx-exporter
```



nginx



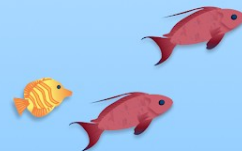
NSD



- Pamiętajcie serwer autorytatywny DNS z poprzedniego tygodnia?
 - On ma wbudowany expoter
- Tylko dashboarda nie ma...



NSD



```
server:  
  metrics-enable: yes  
  metrics-interface: 0.0.0.0  
  metrics-port: 9091
```

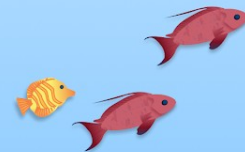
```
root@debian:~# systemctl restart nsd
```

```
- targets: [ 'localhost:9100' ]  
- job_name: 'nsd'  
  static_configs:  
    - targets: [ '192.168.122.80:9091', '192.168.122.81:9091' ]
```

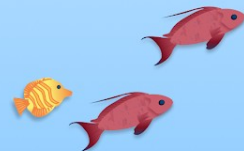
```
root@debian:~# systemctl reload prometheus  
root@debian:~#
```



NSD



Własne dane?



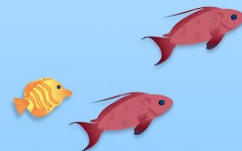
- Format metryk prometheusa jest dość prosty

```
1 # HELP dearrow_queue_length Current length of the queues
2 # TYPE dearrow_queue_length gauge
3 # HELP dearrow_queue_scheduled Current amount of scheduled jobs from the queue
4 # TYPE dearrow_queue_scheduled gauge
5 dearrow_queue_length{queue="high"} 654
6 dearrow_queue_scheduled{queue="high"} 0
7 dearrow_queue_length{queue="low"} 5001
8 dearrow_queue_scheduled{queue="low"} 0
```

- Można albo wystawić serwerem HTTP...
- Albo wrzucić plik .prom do
/var/lib/prometheus/node-exporter



Własne dane?



```
## HELP some_metric Represents some metric
# TYPE some_metric gauge
some_metric{service="a"} 2137
some_metric{service="b"} 67
/var/lib/prometheus/node-exporter/some.prom
```

Metrics explorer ×

Give feedback

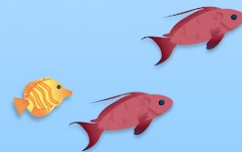
Name	Type	Description
some_metric	gauge	Represents some metric

Raw

Expand results ☐ Result series: 2

some_metric{instance="localhost:9100", job="node", service="a"}	2137
some_metric{instance="localhost:9100", job="node", service="b"}	67



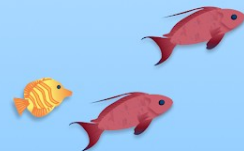


Rozdział 4

Kwerendy PromQL



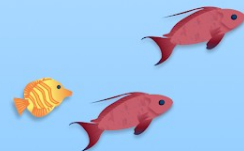
PromQL



- PromQL to sposób na odpytywanie Prometheusa o metryki
- Przydatna jest tu strona Explore na grafanie
- Dokumentacja: prometheus.io/docs/prometheus/latest/querying/basics/



Najprostsze kwerendy



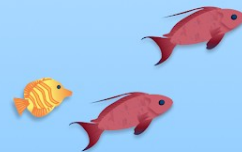
- Najprostsza kwerenda to po prostu nazwa metryki

The screenshot shows the Prometheus Metrics browser interface. At the top, there's a dropdown menu with 'A' and '(prometheus)'. Below it, an 'Explain' toggle is turned off. A 'Metrics browser >' button is visible, followed by the query 'up'. The main display area shows the 'Raw' view of the query results. It includes an 'Expand results' toggle and a 'Result series: 5' indicator. The results are displayed in a table with two columns: the metric name and its value.

up{instance="localhost:9090", job="prometheus"}	1
up{instance="localhost:9100", job="node"}	1
up{instance="192.168.122.80:9091", job="nsd"}	1
up{instance="192.168.122.81:9091", job="nsd"}	1
up{instance="127.67.21.37:9093", job="nginx"}	1



Najprostsze kwerendy



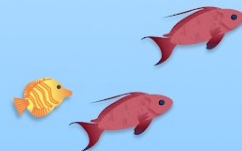
- Możemy wyniki zawęzić, podając etykiety

The screenshot shows the Prometheus Metrics browser interface. At the top, there's a dropdown menu with 'A' and '(prometheus)'. Below it, an 'Explain' toggle is turned off. A 'Metrics browser >' button is visible. The query input field contains 'up{job="nsd"}'. Below the query, there are two tabs: 'Table' and 'Raw', with 'Raw' selected. The 'Raw' view shows two result series, each with a value of 1. The 'Expand results' toggle is also turned off.

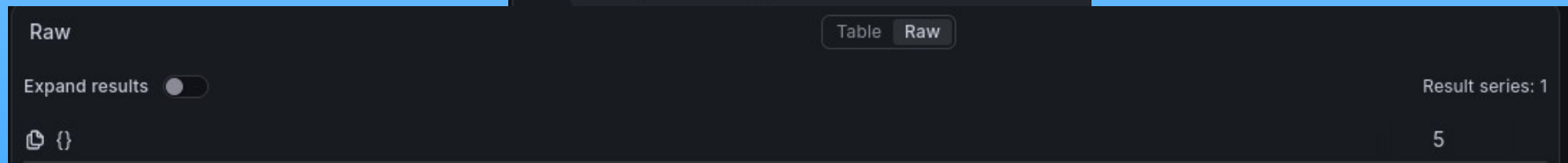
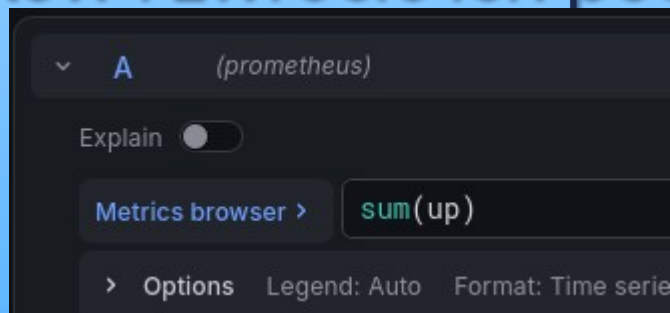
Raw	Table	Raw
up{instance="192.168.122.80:9091", job="nsd"}	1	
up{instance="192.168.122.81:9091", job="nsd"}	1	



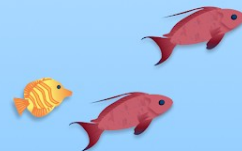
Funkcje agregujące



- Funkcjami agregującymi możemy operować na zbiorze wyników i zwrócić ich podsumowanie



Funkcje agregujące



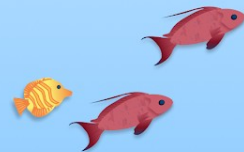
- Można również grupować według etykiet

The screenshot shows the Prometheus Metrics browser interface. At the top, a dropdown menu is set to 'A' with the label '(prometheus)'. Below it, there is an 'Explain' toggle switch. A button labeled 'Metrics browser >' is visible. The main query input field contains the text 'sum by (job) (up)'. Below the query input, there are two tabs: 'Table' and 'Raw', with 'Raw' being the active tab. The 'Raw' view shows a table with four rows of data. The first column contains labels in curly braces, and the second column contains numerical values. The text 'Expand results' is on the left, and 'Result series: 4' is on the right.

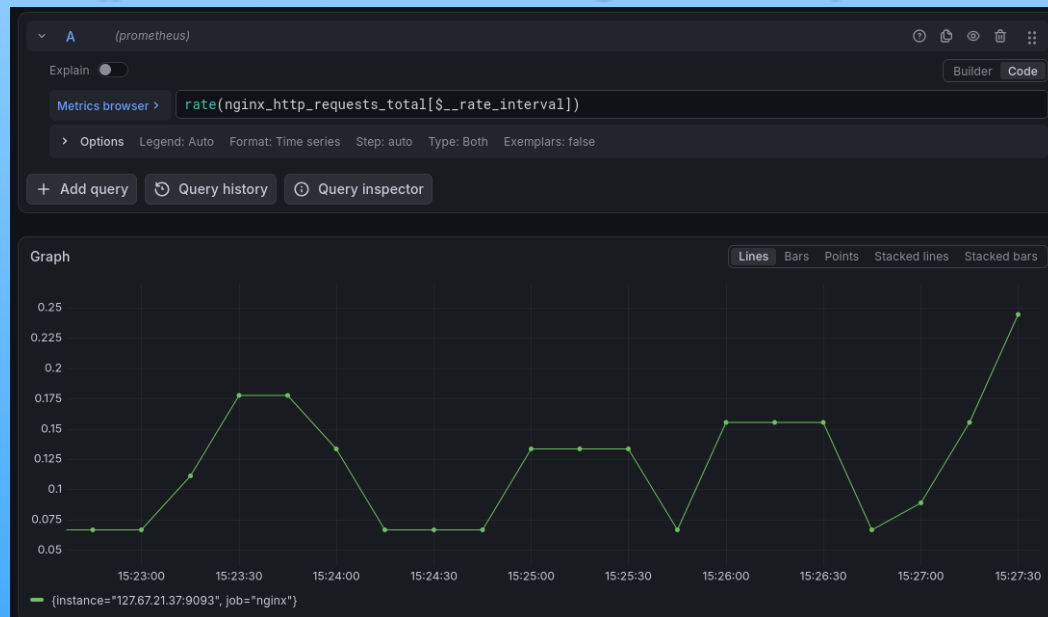
Label	Value
{job="prometheus"}	1
{job="node"}	1
{job="nsd"}	2
{job="nginx"}	1



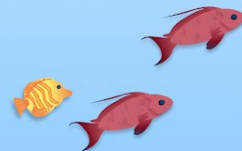
Obliczanie zmian



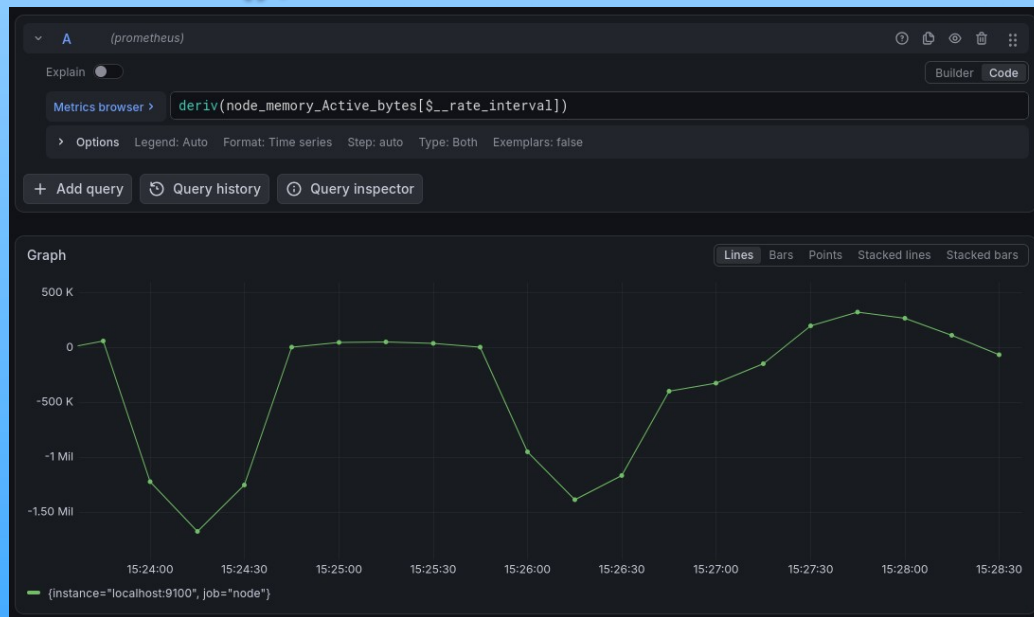
- Funkcją `rate()` można obliczyć tempo zmian liczników



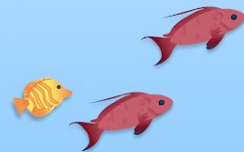
Obliczanie zmian



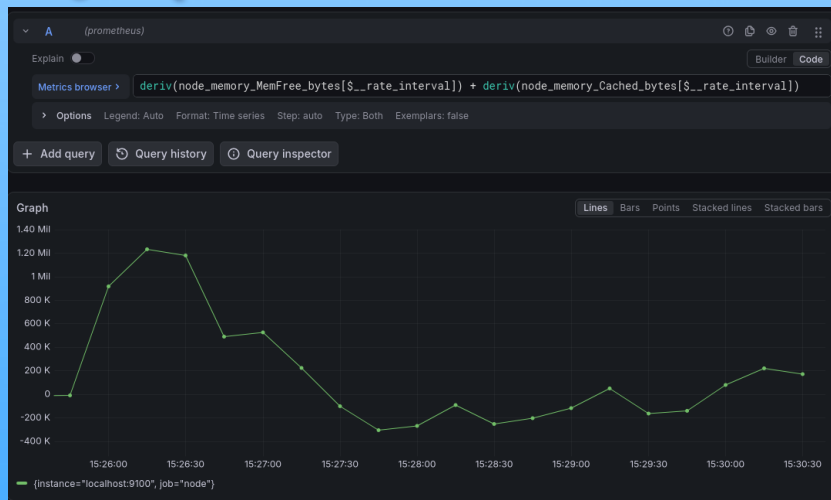
- `deriv()` to `rate()`, ale dla mierników



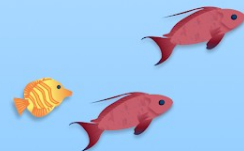
Łączenie metryk



- Tak, metryki można sumować, odejmować, etc.
 - O ile etykiety są takie same



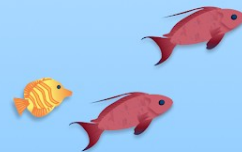
Zmienne



- Zmienne pozwalają użytkownikowi wybrać konkretną instancję serwisu do wyświetlenia
- Wartości zmiennych można generować automatycznie, z wartości etykiet



Zmienne



Query variable: instance

Preview of values (2)

192.168.122.80:9091

192.168.122.81:9091

Query

Static options (0)

Target data source

 `${datasource}`

Query type

Label values

Label *

instance

Metric

up

Label filters

job

=

nsd

x

+

`A` `(${datasource})`

Explain

Metrics browser >

`rate(nsd_queries_total{instance="${instance}"}[${rate_interval}])`

Run queries

> Options

Legend: Server #`{(server)}`

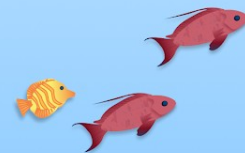
Format: Time series

Step: auto

Type: Range

Exemplars: false



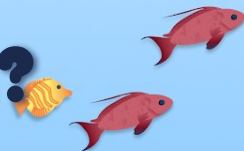


Rozdział 5

Tworzenie alertów



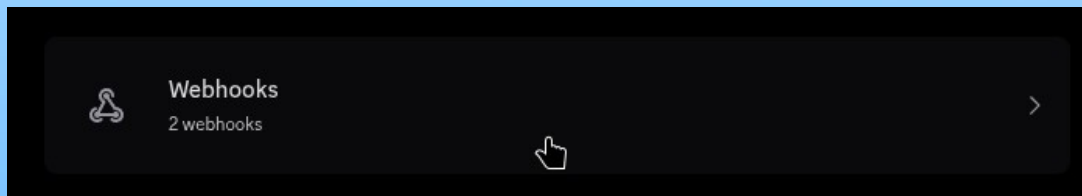
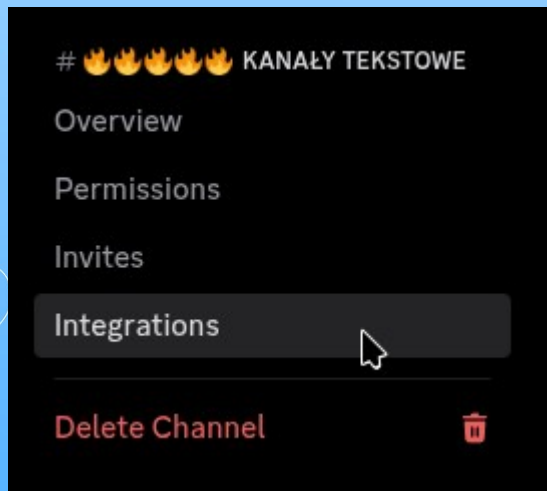
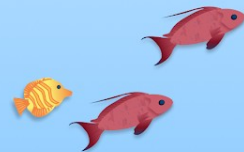
Gdzie chcemy wysyłać powiadomienia?



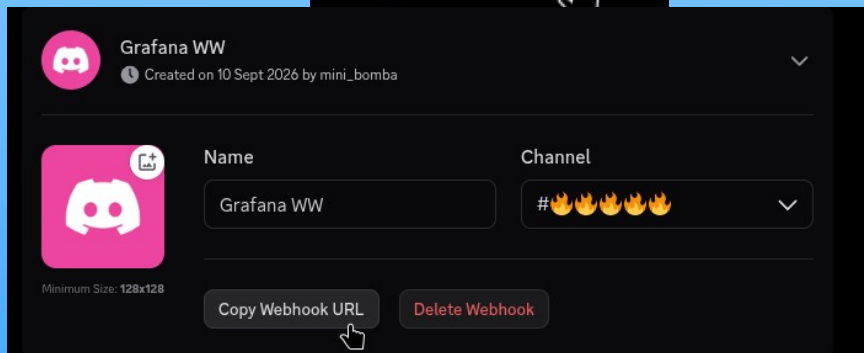
- Grafana ma wiele metod powiadamiania
 - Od maila, przez Teamsy/Slack, aż do webhooków
 - Discorda najprościej skonfigurować



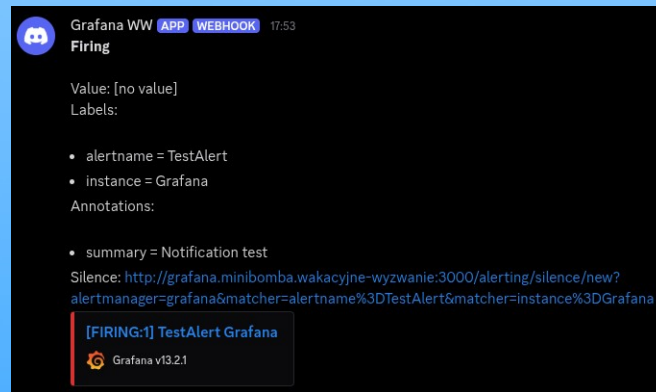
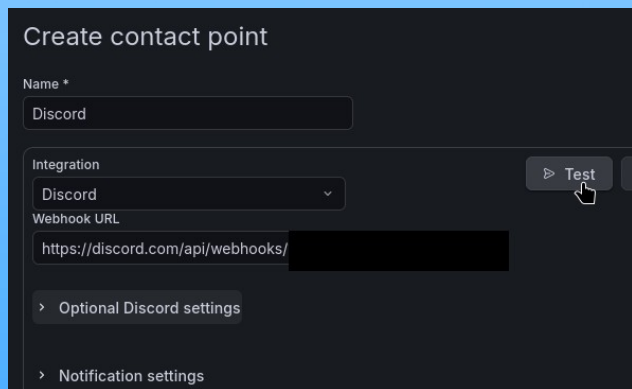
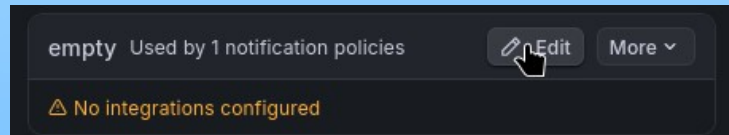
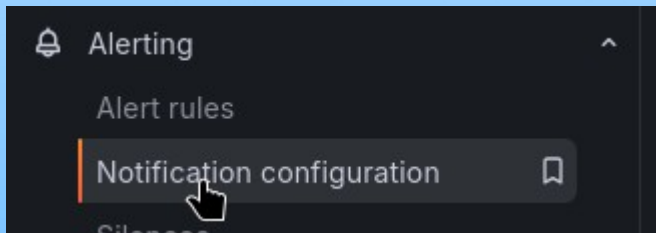
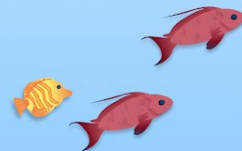
Konfigurowanie powiadomień



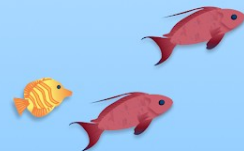
New Webhook



Konfigurowanie powiadomień



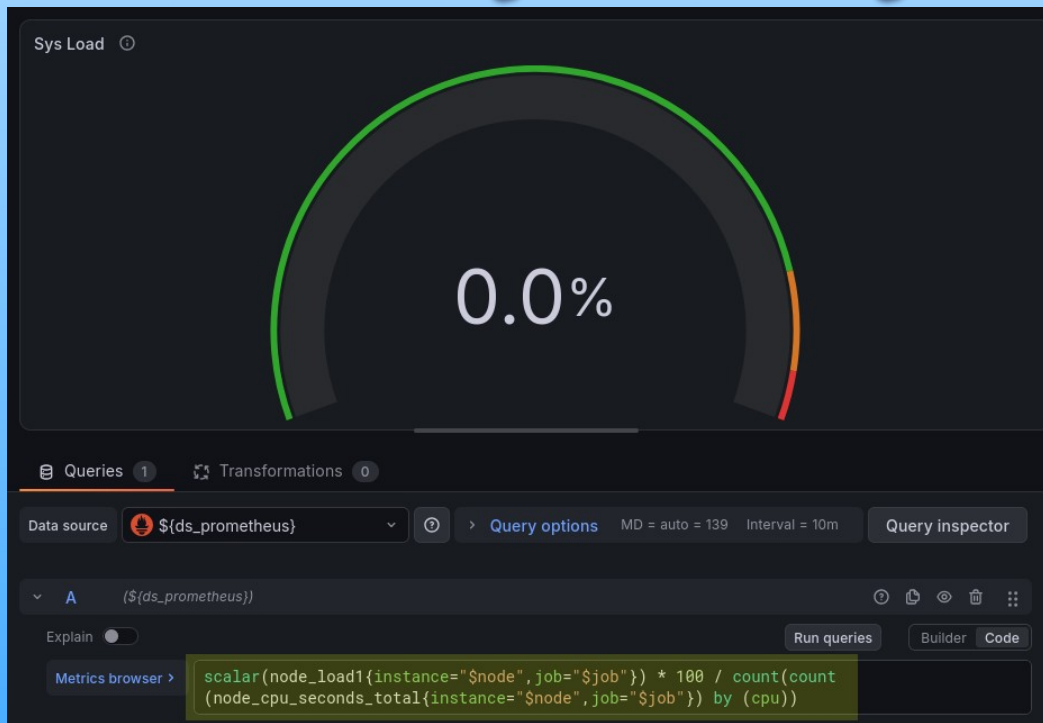
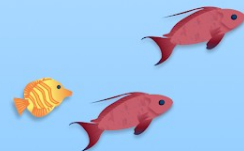
Tworzenie reguł alertowania



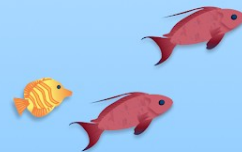
- By utworzyć regułę, trzeba najpierw wiedzieć, na podstawie jakiej metryki chcemy ostrzegać
 - Metryki można poznać w dokumentacji exporterów, w zakładce Explore Grafany, lub sprawdzając gotowe dashboardy
- Reguły ostrzegania powinny zwracać wyniki dla całej grupy monitorowanych instancji
 - Nie trzeba tworzyć reguł dla każdej instancji z osobna



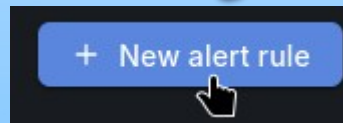
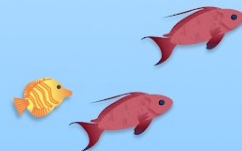
Tworzenie reguł ostrzegania



Tworzenie reguł ostrzegania



Tworzenie reguł ostrzegania



New alert rule

1. Enter alert rule name
Enter a name to identify your alert rule.

Name
LoadAvg over CPU count

2. Define query and alert condition Advanced options
Define query and alert condition [Need help?](#)

prometheus Options 10m to now

Explain Run queries Builder Code

Metrics browser > `node_load1{job="node"} / count(count(node_cpu_seconds_total{job="node"}) by (cpu, instance, job)) by (instance, job)`

Selected metric is a counter. Consider calculating rate of counter by adding rate().

> Options Legend: Auto Format: Time series Step: auto Type: Instant

Table

<code>{instance="localhost:9100", job="node"}</code>	0
--	---

Alert condition

WHEN QUERY IS ABOVE

`{instance="localhost:9100", job="node"}` 0 Normal

[Preview alert rule condition](#)

New evaluation group

Create a new evaluation group to use for this alert rule.

Evaluation group name
A group evaluates all its rules over the same evaluation interval.

Default group

Evaluation interval
How often all rules in the group are evaluated.

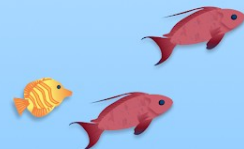
30s

10s 30s 1m 5m 10m 15m 30m 1h

Cancel Create



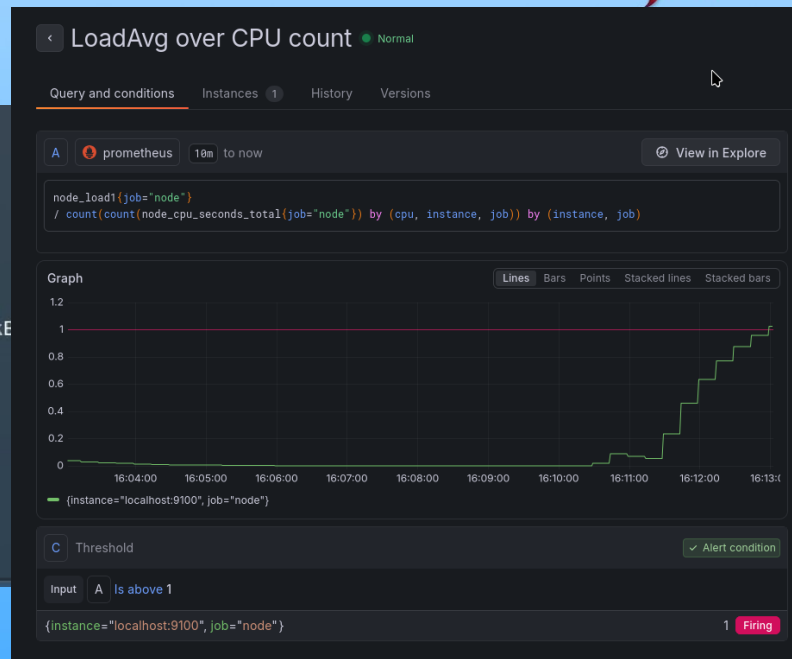
Testowanie reguł ostrzegania



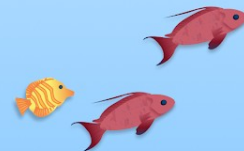
```
root@debian:~# apt install stress
Installing:
  stress

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 0
  Download size: 21.9 kB
  Space needed: 57.3 kB / 13.4 GB available

Get:1 http://ftp.psnc.pl/debian trixie/main amd64 stress amd64 1.0.7-1 [21.9 kB]
Fetched 21.9 kB in 0s (102 kB/s)
Selecting previously unselected package stress.
(Reading database ... 41489 files and directories currently installed.)
Preparing to unpack .../stress_1.0.7-1_amd64.deb ...
Unpacking stress (1.0.7-1) ...
Setting up stress (1.0.7-1) ...
Processing triggers for man-db (2.13.1-1) ...
root@debian:~# stress -c 5
stress: info: [8791] dispatching hogs: 5 cpu, 0 io, 0 vm, 0 hdd
```



Testowanie reguł ostrzegania



 Grafana **APP** **WEBHOOK** 18:14

Firing

Value: A=1.185, C=1

Labels:

- alertname = LoadAvg over CPU count
- grafana_folder = Alerts
- instance = localhost:9100
- job = node

Annotations:

Source: <http://grafana.minibomba.wakacyjne-wyzwanie:3000/alerting/grafana/bfxujv5vrtdsa/view?orgId=1>

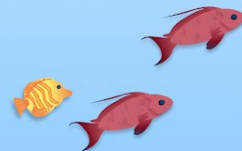
Silence: http://grafana.minibomba.wakacyjne-wyzwanie:3000/alerting/silence/new?alertmanager=grafana&matcher=__alert_rule_uid__%3Dbfxujv5vrtdsa&matcher=instance%3Dlocalhost%3A9100&matcher=job%3Dnode&orgId=1

[FIRING:1] LoadAvg over CPU count Alerts (localhost:9100 node)

 Grafana v13.2.1



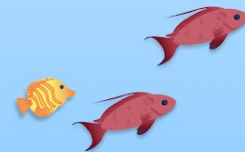
Inne dobre reguły ostrzegania

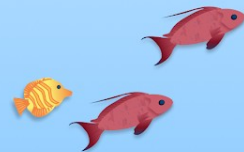


- **Wbudowane:**
 - Niepowodzenie zbierania metryk (metryka up = 0)
- **Node exporter:**
 - Jakaś partycja jest prawie pełna
 - Wysokie zużycie RAMu/swapu
 - Duży ruch na sieci przez długi czas
 - Wysoka temperatura na którymś czujniku
- **Reverse proxy/serwisy:**
 - Nadzwyczajnie duża liczba żądań
- **Blackbox exporter:**
 - Monitorowany serwis/host jest niedostępny



Pytania?





**Dziękuję
za uwagę**

